Universidad de La Laguna

Platform Game with MVC in TypeScripts

Course 2025-2026

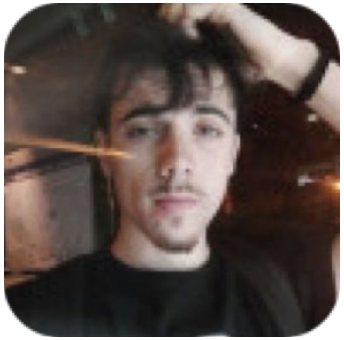
Platform Game in Typescript

Link to code in these slides

Team

Adrián Castro Rodríguez

adrian.castro.46@ull.edu.es



Ezequiel Juan Canale Oliva

ezequiel.juan.11@ull.edu.es

Bruno Morales Hernández

morales.hernandez.28@ull.edu.es

Topics

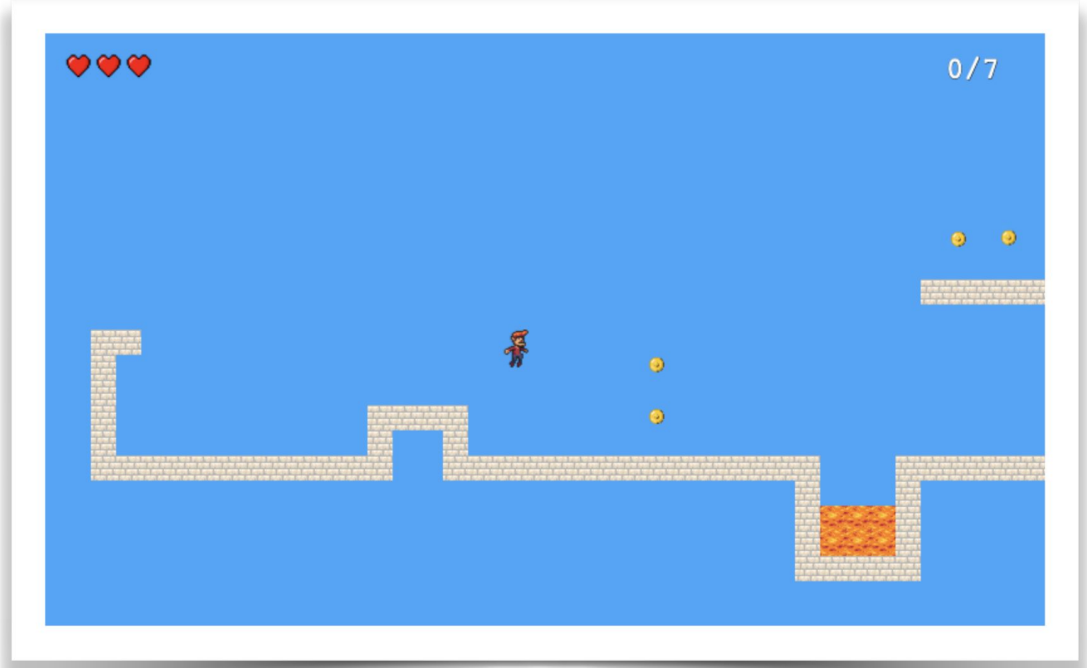
- Original Code
- Live Game Demo
- Legacy Code Analysis
- Transformation Process
- Evolution Code
- MVC Pattern
- Model
- View
- Controller
- Quality Standards & SOLID
- Conclusions

Original Project

Live Demo

Game Overview

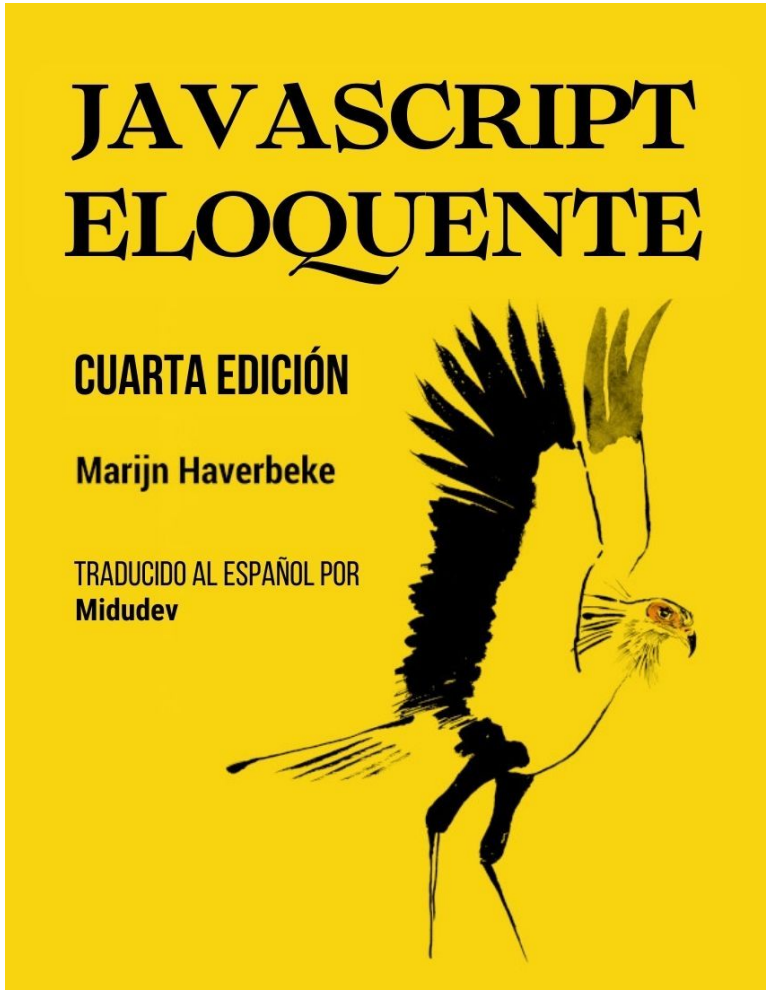
- Platformer game with active physics.
- Core mechanics: Gravity and collisions.
- Main Objective: Collect all coins.



Project Origin

Eloquent JavaScript (Chapter 16)

Origin: 10 year old repository



Legacy Code Challenge

- Monolithic architecture (Single file).
- Untyped JavaScript.
- Obsolete toolchain (Gulp).

Transformation

- Divide in different files

```
'use strict';

import { Vector } from "./Vector.js";

export class Lives {
  constructor(pos) {
    this.pos = pos;
    this.size = new Vector(1, 1);
    this.type = "lives";
  }

  act() {}
}
```

```
'use strict';

import { Vector } from "./Vector.js";

export class Lava {
  constructor(pos, ch) {
    this.pos = pos;
    this.size = new Vector(1, 1);
    this.type = "lava";

    if (ch === "=") {
      this.speed = new Vector(2, 0);
    } else if (ch === "|") {
      this.speed = new Vector(0, 2);
    } else if (ch === "v") {
      this.speed = new Vector(0, 3);
      this.repeatPos = pos;
    }
  }

  act(step, level) {
    const newPos = this.pos.plus(this.speed.times(step));
    if (!level.obstacleAt(newPos, this.size)) {
      this.pos = newPos;
    } else if (this.repeatPos) {
      this.pos = this.repeatPos;
    } else {
      this.speed = this.speed.times(-1);
    }
  }
}
```

```
'use strict';

import { Vector } from "./Vector.js";

export class Score {
  constructor(pos) {
    this.pos = pos;
    this.size = new Vector(1, 1);
    this.type = "score";
  }

  act() {}
}
```

Transformation

- Translate to TypeScript

```typescript
import { Level } from "./Level.js";
import { Vector } from "./Vector.js";
import { Actor, ActorType, KeyMap } from "./Actor.js";

export class Coin implements Actor {
  pos: Vector;
  basePos: Vector;
  size: Vector;
  wobble: number;
  type: ActorType = "coin";

  private static readonly WOBBLE_SPEED = 8;
  private static readonly WOBBLE_DIST = 0.07;

  constructor(pos: Vector) {
    this.basePos = this.pos = pos.plus(new Vector(0.2, 0.1));
    this.size = new Vector(0.6, 0.6);
    this.wobble = Math.random() * Math.PI * 2;
  }

  act(step: number, _level: Level, _keys: KeyMap): void {
    this.wobble += step * Coin.WOBBLE_SPEED;
    const wobblePos = Math.sin(this.wobble) * Coin.WOBBLE_DIST;
    this.pos = this.basePos.plus(new Vector(0, wobblePos));
  }
}
```

```typescript
import { Level } from "./Level.js";
import { Vector } from "./Vector.js";
import { Actor, ActorType, KeyMap } from "./Actor.js";

export class Lava implements Actor {
  pos: Vector;
  size: Vector;
  speed: Vector;
  repeatPos?: Vector;
  type: ActorType = "lava";

  constructor(pos: Vector, char: string) {
    this.pos = pos;
    this.size = new Vector(1, 1);

    if (char === "=") {
      this.speed = new Vector(2, 0);
    } else if (char === "|") {
      this.speed = new Vector(0, 2);
    } else if (char === "v") {
      this.speed = new Vector(0, 3);
      this.repeatPos = pos;
    } else {
      this.speed = new Vector(0, 0);
    }
  }

  act(step: number, level: Level, keys: KeyMap): void {
    const newPos = this.pos.plus(this.speed.times(step));
    if (!level.obstacleAt(newPos, this.size)) {
      this.pos = newPos;
    } else if (this.repeatPos) {
      this.pos = this.repeatPos;
    } else {
      this.speed = this.speed.times(-1);
    }
  }
}
```

Transformation

- Convert to MVC

```
/** @classdesc Owns the mutable game state and exposes the minimal surface. */
> export class GameModel {…
}
```

```
/** @classdesc Single entry-point to the view layer. */
> export class GameView {…
}
```

```
/** @classdesc Acts as the central controller in an MVC architecture. */
> export class GameController {…
}
```

Structure

Structural Evolution

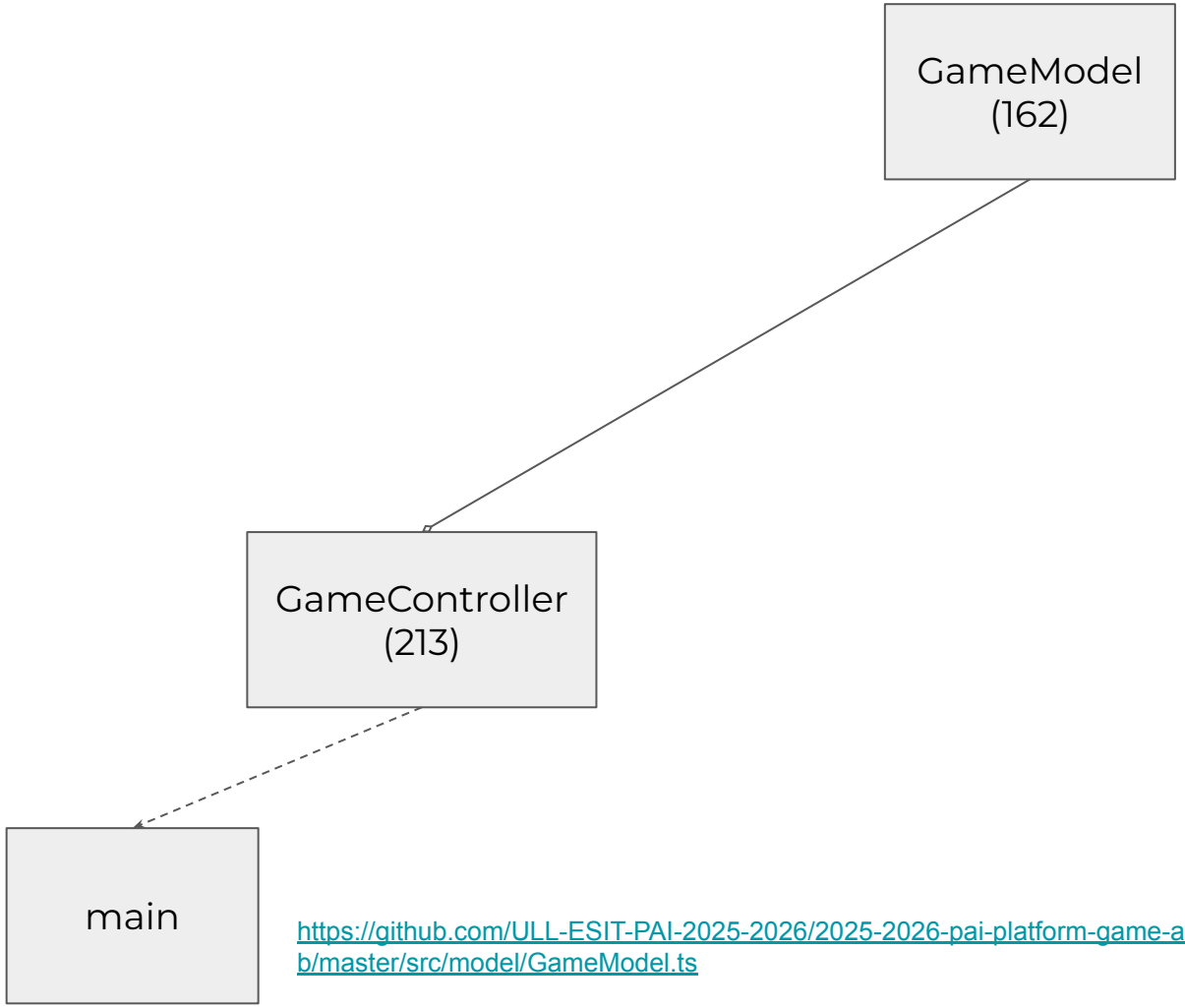
```
├── assets
│   ├── img
│   │   ├── lives.png
│   │   ├── player.png
│   │   └── sprites.png
│   ├── scripts
│   │   ├── game.js
│   │   └── game_levels.js
│   ├── sound
│   │   ├── 201-overworld-bgm.mp3
│   │   ├── smb_coin.wav
│   │   ├── smb_fireworks.wav
│   │   ├── smb_gameover.wav
│   │   ├── smb_jump-small.wav
│   │   ├── smb_mariodie.wav
│   │   ├── smb_stage_clear.wav
│   │   └── smb_world_clear.wav
│   └── styles
│       └── game.css
├── gulpfile.js
├── index.html
├── package-lock.json
└── package.json
```

```
src/
├── GameLevels.ts
├── controller
│   └── GameController.ts
├── main.ts
├── model
│   ├── Actor.ts
│   ├── Coin.ts
│   ├── GameModel.ts
│   ├── Lava.ts
│   ├── Level.ts
│   ├── Lives.ts
│   ├── Player.ts
│   └── Score.ts
├── vector.ts
└── view
    ├── ActorView.ts
    ├── CanvasDisplay.ts
    ├── GameView.ts
    └── PlayerView.ts
```

Structure explanation

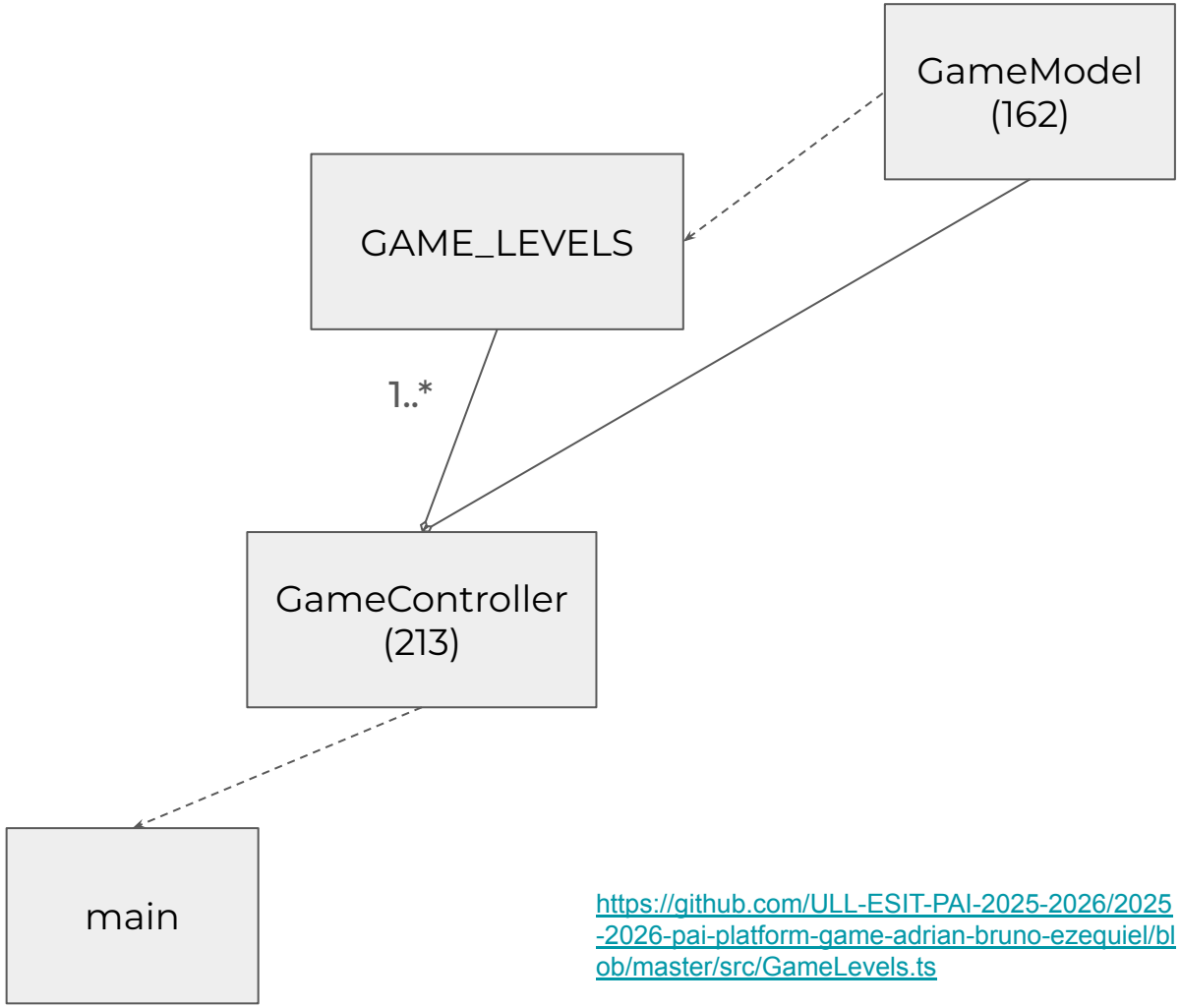
```html
<html>
  <head>
    <title>Plataform Game</title>
    <meta charset="utf-8">
    <meta name="author" content="Adrián Castro Rodríguez">
    <meta name="author" content="Bruno Morales Hernández">
    <meta name="author" content="Ezequiel Juan Canale Oliva">
    <link rel="stylesheet" href="assets/styles/styles.css">
    <script type="module" src="main.js"></script>
  </head>
  <body>
    <audio id="music"></audio>
    <audio id="sound"></audio>
    <canvas id = "main-canvas"></canvas>
  </body>
</html>
```

Structure explanation



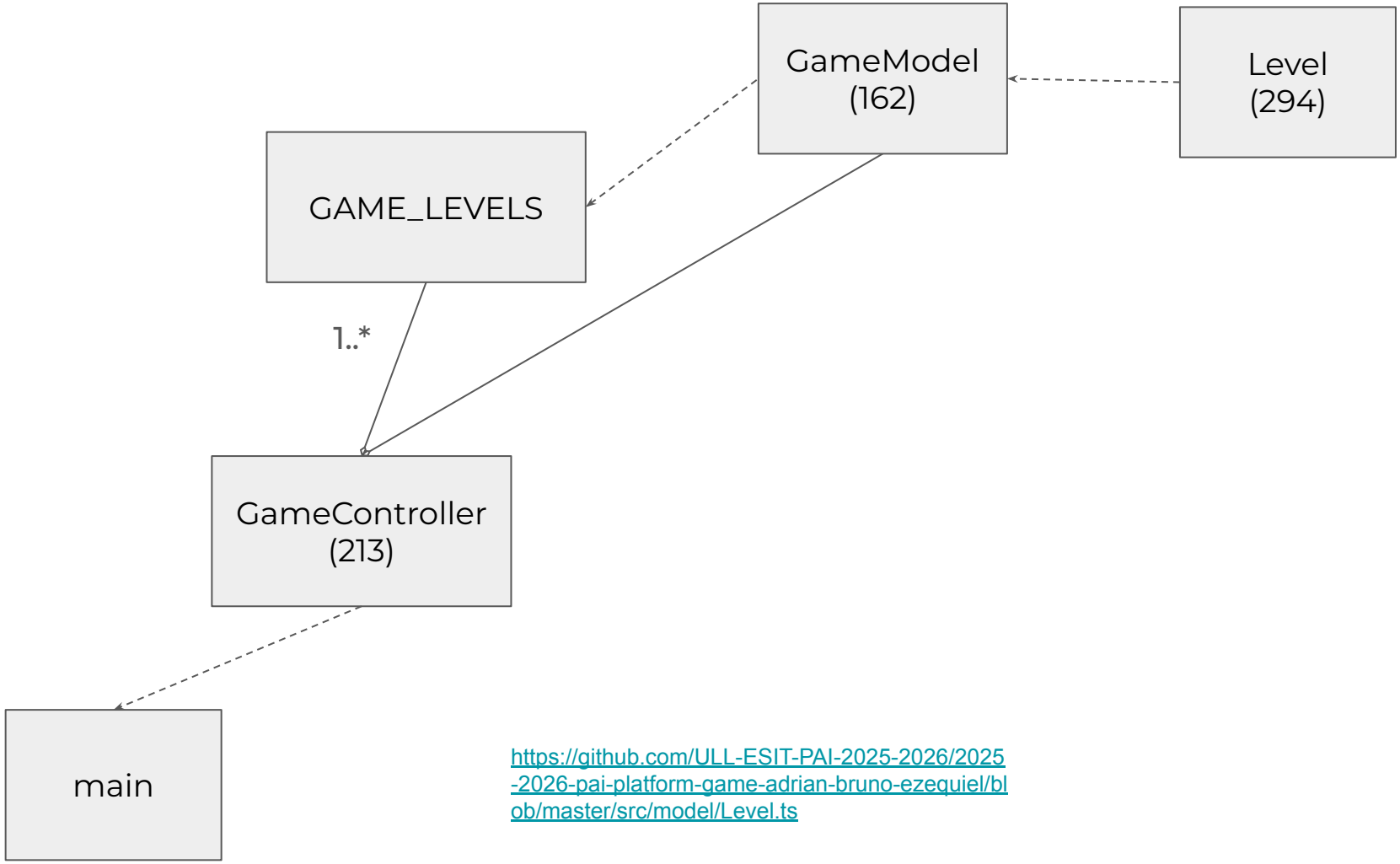
https://github.com/ULL-ESIT-PAI-2025-2026/2025-2026-pai-platform-game-adrian-bruno-ezequiel/blob/master/src/model/GameModel.ts

Structure explanation



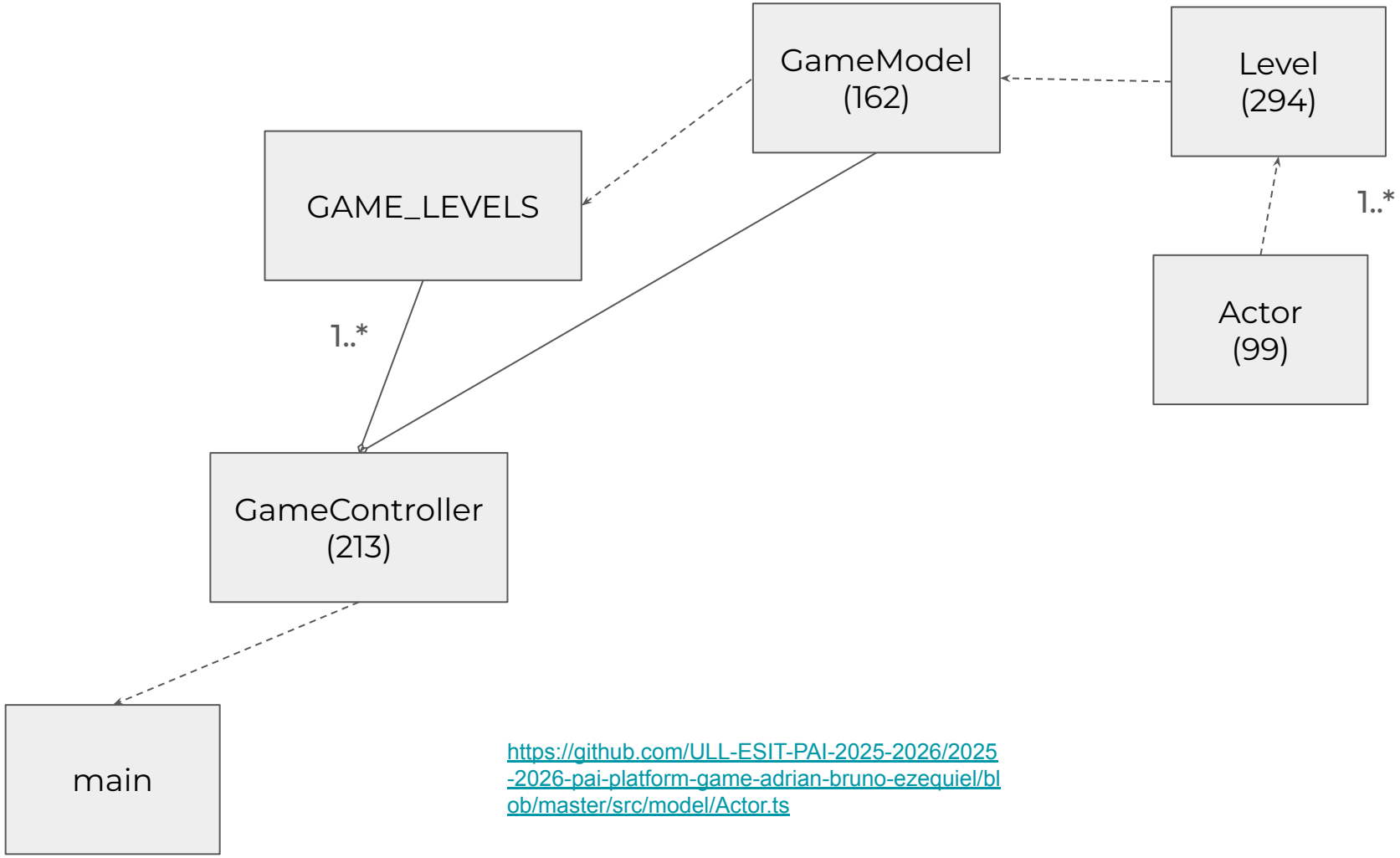
https://github.com/ULL-ESIT-PAI-2025-2026/2025-2026-pai-platform-game-adrian-bruno-ezequiel/blob/master/src/GameLevels.ts

Structure explanation



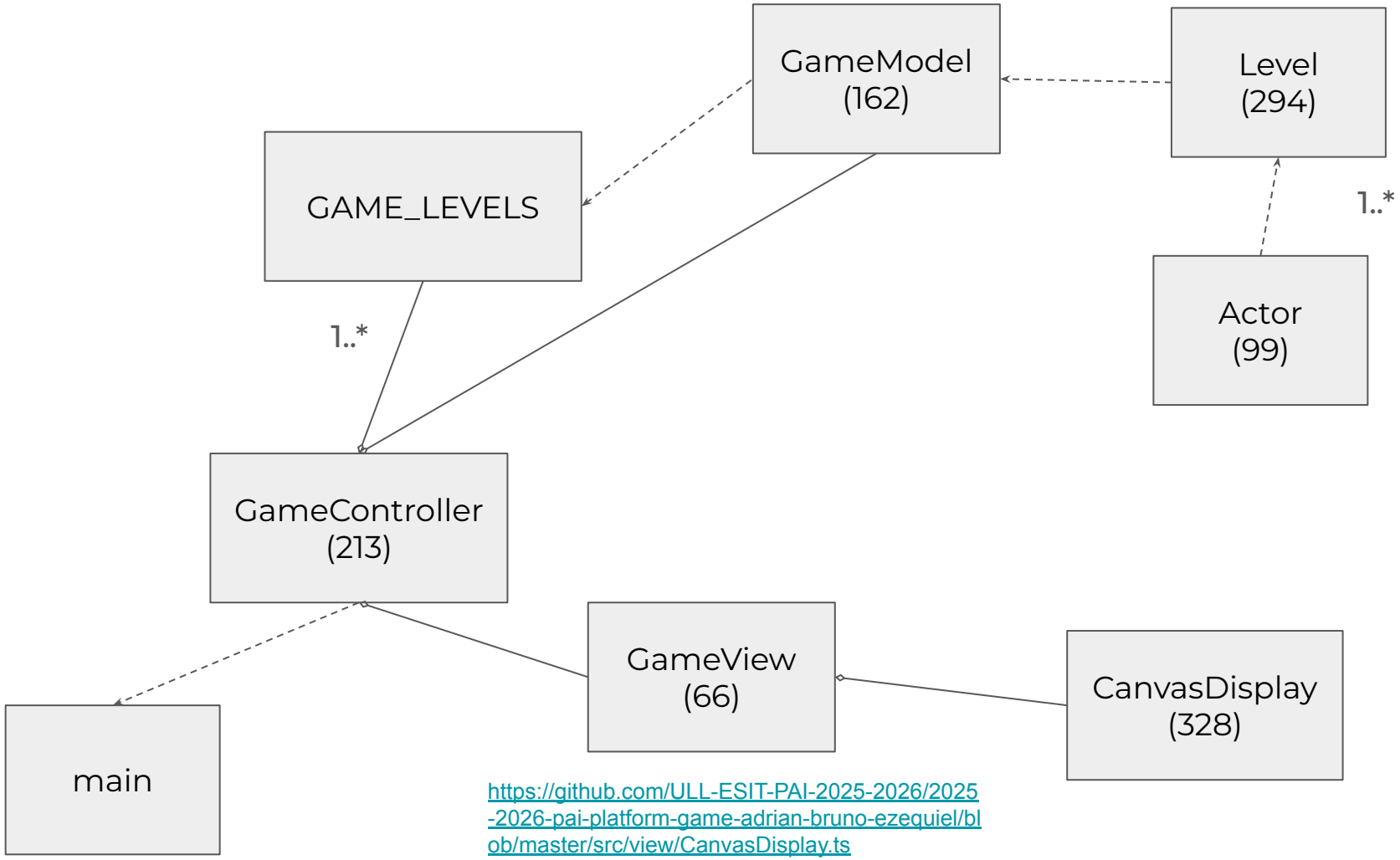
https://github.com/ULL-ESIT-PAI-2025-2026/2025-2026-pai-platform-game-adrian-bruno-ezequiel/blob/master/src/model/Level.ts

Structure explanation



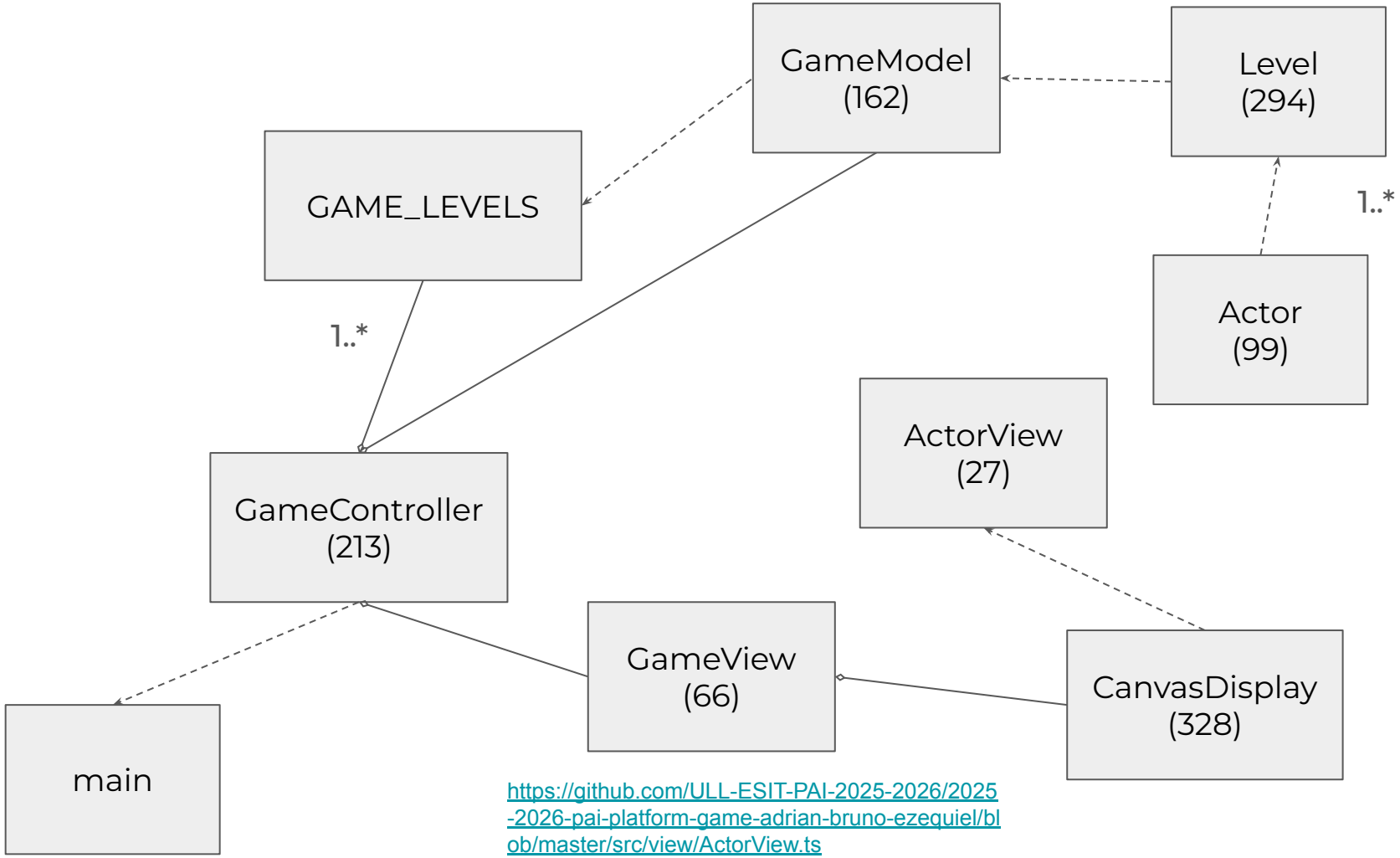
https://github.com/ULL-ESIT-PAI-2025-2026/2025-2026-pai-platform-game-adrian-bruno-ezequiel/blob/master/src/model/Actor.ts

Structure explanation



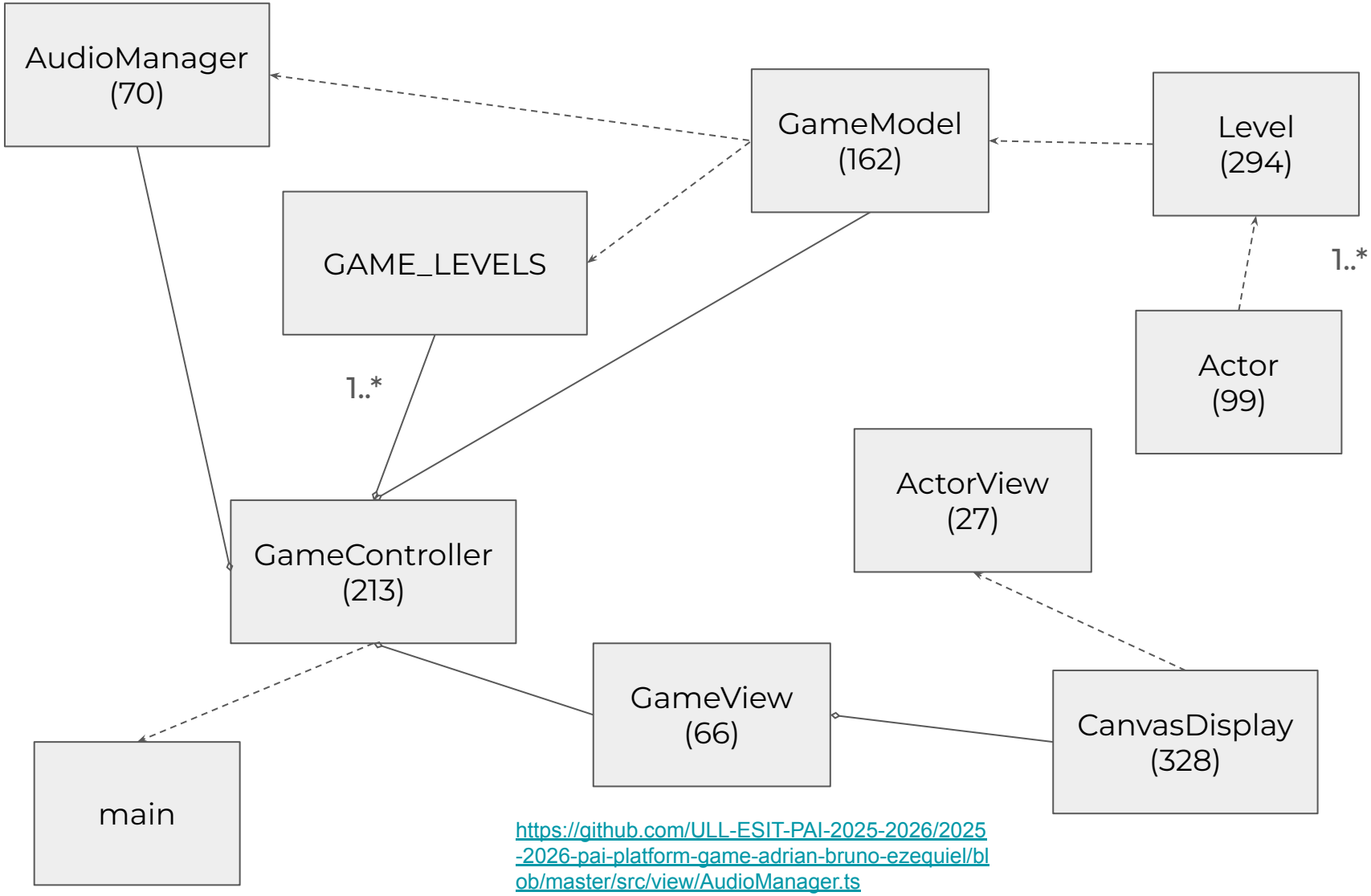
https://github.com/ULL-ESIT-PAI-2025-2026/2025-2026-pai-platform-game-adrian-bruno-ezequiel/blob/master/src/view/CanvasDisplay.ts

Structure explanation



https://github.com/ULL-ESIT-PAI-2025-2026/2025-2026-pai-platform-game-adrian-bruno-ezequiel/blob/master/src/view/ActorView.ts

Structure explanation



https://github.com/ULL-ESIT-PAI-2025-2026/2025-2026-pai-platform-game-adrian-bruno-ezequiel/blob/master/src/view/AudioManager.ts

Game Model

GameModel

- The brain of the application.
- Manages global state (Lives, Levels).
- Acts as the bridge between the Controller and the Level.

GameModel.ts

Why not connect the Controller directly to the Level?

- Persistence vs Volatility
  - Levels are temporary
  - Model is permanent

Why not connect the Controller directly to the Level?

- Persistence vs Volatility
  - Levels are temporary
  - Model is permanent
- Smart Separation
  - The Controller reads the Level directly.
  - To change scenes, the Controller just asks the Model

State and Flow Management

- Level management.

State and Flow Management

- Level management.

```
private currentLevel: Level | null = null;
private currentLevelIndex: number = 0;

loadNextLevel(): Level { /* ... */ }
reloadCurrentLevel(): Level { /* ... */ }
reset(lives: number = 3): Level { /* ... */ }
```

State and Flow Management

- Level management.
- Global Lives Tracking.

State and Flow Management

- Level management.
- Global Lives Tracking.

```
private lives: number;

loseLife(): void {
  this.lives = Math.max(0, this.lives - 1);
  this.currentLevel?.setLives(this.lives);
}
```

State and Flow Management

- Level management.
- Global Lives Tracking.
- Controller-Level Intermediary.

State and Flow Management

- Level management.
- Global Lives Tracking.
- Controller-Level Intermediary.

```
animate(step: number, keys: KeyMap): void {
  this.currentLevel!.animate(step, keys);
}
```

Facade Pattern

- Level management.
- Cleaner imports

Facade Pattern

- Level management.
- Cleaner imports

```
export {Level} from './Level.js';
export {Actor, ActorType, KeyMap} from
'./Actor.js';
export {Vector} from '../Vector.js';
export {Player} from './Player.js';
export {Coin} from './Coin.js';
export {Lava} from './Lava.js';
export {Lives} from './Lives.js';
export {Score} from './Score.js';
```

Resposibilities

```
export class GameModel {
  private levelMap: string[][];
  private currentLevel: Level | null;
  private currentLevelIndex: number;
  private lives: number;
}
```

Resposibilities

```
export class GameModel {
  private levelMap: string[][];
  private currentLevel: Level | null;
  private currentLevelIndex: number;
  private lives: number;
}
```

```
loadLevel(index: number): Level
reloadCurrentLevel(): Level
loadNextLevel(): Level
reset(lives: number): Level
animate(step: number, keys: KeyMap)
loseLife(): void
```

Level

- Interprets a level

Level.ts

Level

- Interprets a level
- Detects collisions

Level.ts

Level

- Interprets a level
- Detects collisions
- Game state

Level.ts

Level

- Interprets a level
- Detects collisions
- Game state
- Events (coins, lives...)

Level.ts

Actor

Actor.ts

Actor

- ‘player’ - The playable character (Player.ts)

Actor.ts

Actor

- ‘player’ - The playable character (Player.ts)
- ‘coin’ - Collectable coins (Coin.ts)

Actor.ts

Actor

- ‘player’ - The playable character (Player.ts)
- ‘coin’ - Collectable coins (Coin.ts)
- ‘lava’ - Hazard that damages the player (Lava.ts)

Actor.ts

Actor

- ‘player’ - The playable character (Player.ts)
- ‘coin’ - Collectable coins (Coin.ts)
- ‘lava’ - Hazard that damages the player (Lava.ts)
- ‘score’ - UI that shows the score (Score.ts)

Actor.ts

Actor

- ‘player’ - The playable character (Player.ts)
- ‘coin’ - Collectable coins (Coin.ts)
- ‘lava’ - Hazard that damages the player (Lava.ts)
- ‘score’ - UI that shows the score (Score.ts)
- ‘lives’ - UI that shows lives left (Lives.ts)

Actor.ts

Game View

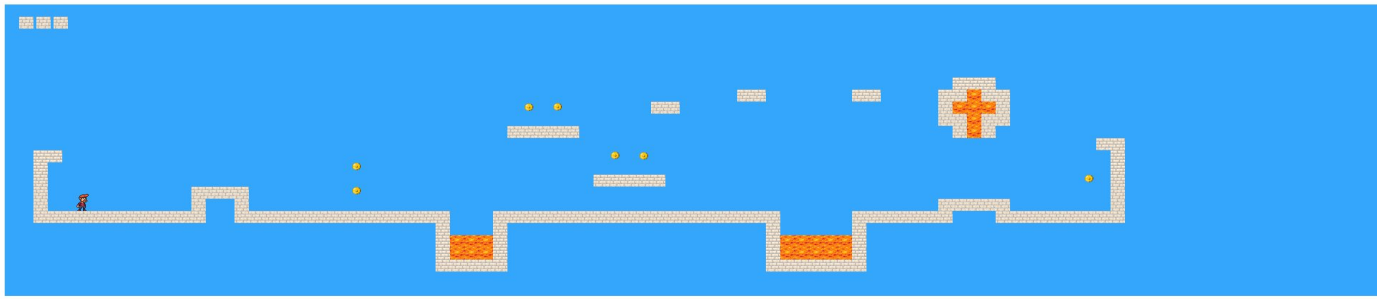
Canvas Display - Cache of images

```
 ...
this.otherSprites = document.createElement('img');
this.otherSprites.src = '/assets/img/sprites.png'
this.playerSprite = document.createElement('img');
this.playerSprite.src = '/assets/img/player.png';
...
```

Canvas Display - Cache of images

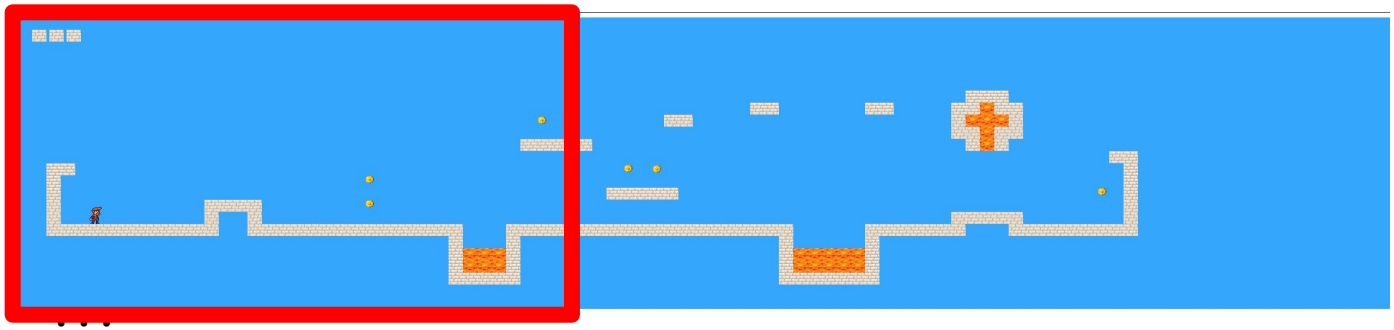
```
 ...
this.otherSprites = document.createElement('img');
this.otherSprites.src = '/assets/img/sprites.png'
this.playerSprite = document.createElement('img');
this.playerSprite.src = '/assets/img/player.png';
 this.viewport = {
  left: 0,
  top: 0,
  width: this.canvas.width / this.scale,
  height: this.canvas.height / this.scale,
 }
```

Canvas Display - Separation of concerns



```
 ...
this.otherSprites = document.createElement(‘img’);
this.otherSprites.src = '/assets/img/sprites.png'
this.playerSprite = document.createElement("img");
this.playerSprite.src = '/assets/img/player.png';
 this.viewport = {
  left: 0,
  top: 0,
  width: this.canvas.width / this.scale,
  height: this.canvas.height / this.scale,
 }
```

Canvas Display - Separation of concerns



```
...
this.otherSprites = document.createElement('img');
this.otherSprites.src = '/assets/img/sprites.png'
this.playerSprite = document.createElement('img');
this.playerSprite.src = '/assets/img/player.png';
 this.viewport = {
  left: 0,
  top: 0,
  width: this.canvas.width / this.scale,
  height: this.canvas.height / this.scale,
 }
```

Canvas Display

```
drawFrame( drawFrameData: DrawFrameData) {
}
```

Canvas Display - DrawFrameData

```
interface drawFrameData {
            step: number,
            center: Vector,
            worldWidth: number, worldHeight: number,
            actors: ActorView[], uiElements: ActorView[],
            playerView: PlayerView,
            numberOfCoins: number,
            numberOfCollectedCoins: number,
            status: Status)
    }
```

Canvas Display - DrawFrameData

```
interface drawFrameData {
            step: number
    }
```

Canvas Display - DrawFrameData

```
interface drawFrameData {
            step: number,
            center: Vector,
            worldWidth: number, worldHeight: number
    }
```

Canvas Display - DrawFrameData

```
interface drawFrameData {
            step: number,
            center: Vector,
            worldWidth: number, worldHeight: number,
            actors: ActorView[], uiElements: ActorView[],
            playerView: PlayerView
    }
```

Canvas Display - DrawFrameData

```
interface drawFrameData {
            step: number,
            center: Vector,
            worldWidth: number, worldHeight: number,
            actors: ActorView[], uiElements: ActorView[],
            playerView: PlayerView,
            numberOfCoins: number,
            numberOfCollectedCoins: number
    }
```

Canvas Display - DrawFrameData

```
interface drawFrameData {
            step: number,
            center: Vector,
            worldWidth: number, worldHeight: number,
            actors: ActorView[], uiElements: ActorView[],
            playerView: PlayerView,
            numberOfCoins: number,
            numberOfCollectedCoins: number,
            status: Status)
    }
```

Canvas Display

```
drawFrame( ... ) {
  this.animationTime += step;
  this.updateViewport(center, worldWidth, worldHeight);
  this.clearDisplay(status);
  this.drawBackground(grid);
  this.drawActor(actors, uiElements,
                 numberOfCoins, numberOfCollectedCoins);
  this.drawPlayer(playerView);
}
```

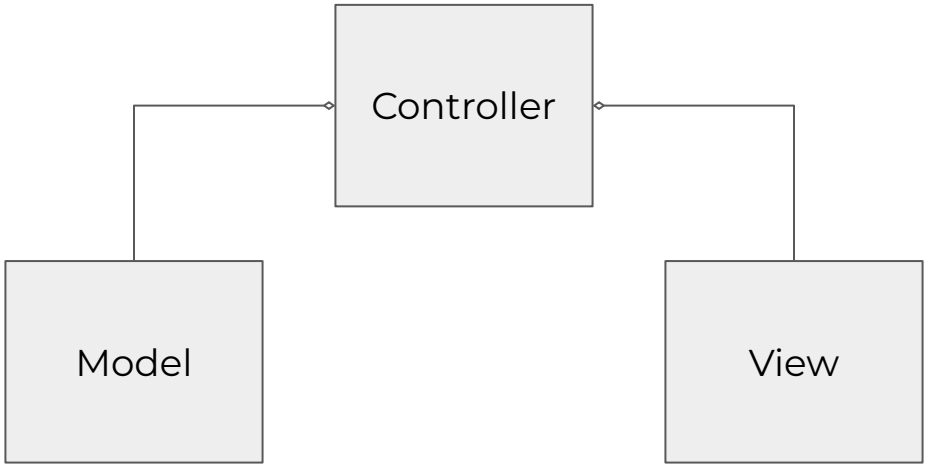
Canvas Display

```
worldToCanvas(pos: Vector): Vector {
  return new Vector(
    (pos.x - this.viewport.left) * this.scale,
    (pos.y - this.viewport.top) * this.scale
  );
}
```

Controller

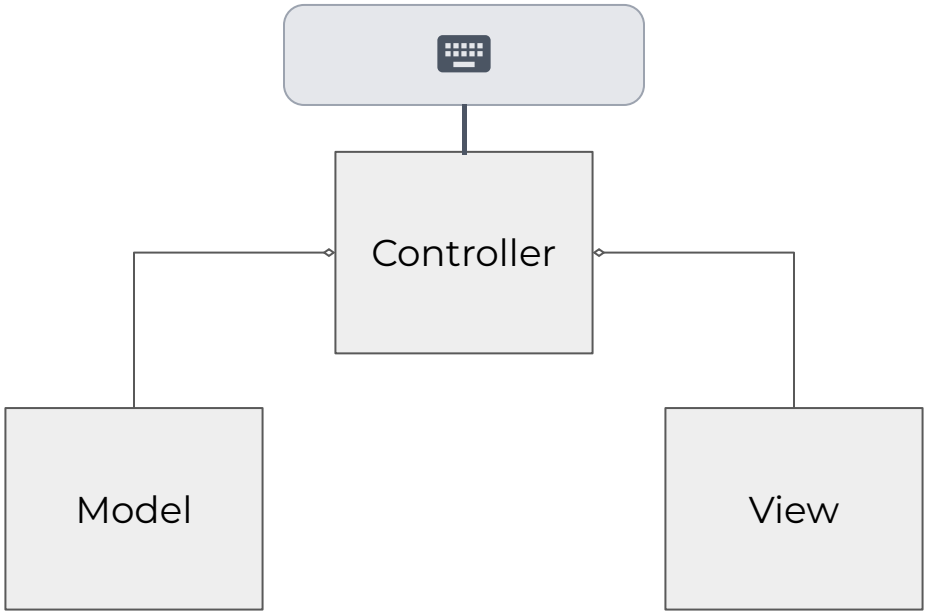
GameController

- Connects the GameModel and the GameView



GameController

- Connects the GameModel and the GameView
- Input reader



GameController - Dashboard

```
private readonly model: GameModel;
private readonly view: GameView;
```

GameController - Dashboard

```
private readonly model: GameModel;
private readonly view: GameView;

private lastTime: number | null = null;
private animationFrameId: number | null = null;
```

GameController - Dashboard

```
private readonly model: GameModel;
private readonly view: GameView;

private lastTime: number | null = null;
private animationFrameId: number | null = null;

private readonly keys: KeyMap = Object.create(null);
```

GameController - Dashboard

```
private readonly model: GameModel;
private readonly view: GameView;

private lastTime: number | null = null;
private animationFrameId: number | null = null;

private readonly keys: KeyMap = Object.create(null);
```

```
{ [key: string]: boolean }
```

GameController - Dashboard

```
private readonly model: GameModel;
private readonly view: GameView;

private lastTime: number | null = null;
private animationFrameId: number | null = null;

private readonly keys: KeyMap = Object.create(null);

private readonly audioManager: AudioManager;
```

GameController - Constructor

```
constructor(model: GameModel, view: GameView) {
  this.model = model;
  this.view = view;

  this.setupKeyTracking();

  this.audioManager = new AudioManager();
  globalThis.addEventListener('keydown', () => {
    this.audioManager.playBackground();
  }, {once: true});
}
```

GameController - Constructor

```
constructor(model: GameModel, view: GameView) {
  this.model = model;
  this.view = view;

  this.setupKeyTracking();

  this.audioManager = new AudioManager();
  globalThis.addEventListener('keydown', () => {
    this.audioManager.playBackground();
  }, {once: true});
}
```

GameController - Input Management

```
private setupKeyTracking(): void {
  const handler = (event: KeyboardEvent) => {
      // Logic inside the handler...
  };
  globalThis.addEventListener('keydown', handler);
  globalThis.addEventListener('keyup', handler);
  this.keyHandlerCleanup = () => {
    globalThis.removeEventListener('keydown', handler);
    globalThis.removeEventListener('keyup', handler);
  };
}
```

GameController - Input Management

```
private setupKeyTracking(): void {
  const handler = (event: KeyboardEvent) => {
      // Logic inside the handler...
  };
  globalThis.addEventListener('keydown', handler);
  globalThis.addEventListener('keyup', handler);
  this.keyHandlerCleanup = () => {
    globalThis.removeEventListener('keydown', handler);
    globalThis.removeEventListener('keyup', handler);
  };
}
```

GameController - Input Management

```
const handler = (event: KeyboardEvent) => {
  const action = GameController.KEY_MAP[event.code];
  if (!action) return;
  const down = event.type === 'keydown';
  if (action === 'esc') {
    if (!down) {
      this.keys['esc'] = !this.keys['esc'];
    }
  } else {
    this.keys[action] = down;
  }
  event.preventDefault();
};
```

GameController - Input Management

```
const handler = (event: KeyboardEvent) => {
  const action = GameController.KEY_MAP[event.code];
  if (!action) return;
  const down = event.type === 'keydown';
  if (action === 'esc') {
    if (!down) {
      this.keys['esc'] = !this.keys['esc'];
    }
  } else {
    this.keys[action] = down;
  }
  event.preventDefault();
};
```

GameController - Input Management

```
  private static readonly KEY_MAP:
Record<string, string> = {
    'Escape': 'esc',        // Escape key
    'ArrowLeft': 'left',    // Left arrow key
    'ArrowUp': 'up',        // Up arrow key
    'ArrowRight': 'right', // Right arrow key
  };
```

GameController - Input Management

```
  private static readonly KEY_MAP:
Record<string, string> = {
    'Escape': 'esc',        // Escape key
    'ArrowLeft': 'left',    // Left arrow key
    'ArrowUp': 'up',        // Up arrow key
    'ArrowRight': 'right', // Right arrow key
  };
```

GameController - Input Management

- Hardware Abstraction

```
  private static readonly KEY_MAP:
Record<string, string> = {
    'Escape': 'esc',        // Escape key
    'ArrowLeft': 'left',    // Left arrow key
    'ArrowUp': 'up',        // Up arrow key
    'ArrowRight': 'right', // Right arrow key
  };
```

GameController - Input Management

```
const handler = (event: KeyboardEvent) => {
  const action = GameController.KEY_MAP[event.code];
  if (!action) return;
  const down = event.type === 'keydown';
  if (action === 'esc') {
    if (!down) {
      this.keys['esc'] = !this.keys['esc'];
    }
  } else {
    this.keys[action] = down;
  }
  event.preventDefault();
};
```

GameController - Input Management

```
const handler = (event: KeyboardEvent) => {
  const action = GameController.KEY_MAP[event.code];
  if (!action) return;
  const down = event.type === 'keydown';
  if (action === 'esc') {
    if (!down) {
      this.keys['esc'] = !this.keys['esc'];
    }
  } else {
    this.keys[action] = down;
  }
  event.preventDefault();
};
```

GameController - Input Management

```
const handler = (event: KeyboardEvent) => {
  const action = GameController.KEY_MAP[event.code];
  if (!action) return;
  const down = event.type === 'keydown';
  if (action === 'esc') {
    if (!down) {
      this.keys['esc'] = !this.keys['esc'];
    }
  } else {
    this.keys[action] = down;
  }
  event.preventDefault();
};
```

GameController - Initialization

- Public Entry Point

```
start(): void {
  this.startLevel(0, 3);
}
```

GameController - Initialization

```
private startLevel(index: number, lives: number): void
{
  this.stopAnimation();
  const level = this.model.reset(lives);
  const currentLevel = index === 0 ? level :
this.model.loadLevel(index);
  this.view.mount(currentLevel.getWidth(),
currentLevel.getHeight());
  this.runAnimation();
}
```

GameController - Game Loop

- Resets the clock for accurate timing.
- Triggers the first asynchronous frame.

```
private runAnimation(): void {
  this.lastTime = null;
  const frame = (time: number) => {
      // Logic inside the frame...
  };
  this.animationFrameId =
requestAnimationFrame(frame);
}
```

GameController - Game Loop

- Captures the initial timestamp.

```
const frame = (time: number) => {
  if (this.lastTime === null) {
    this.lastTime = time;
    this.animationFrameId =
requestAnimationFrame(frame);
    return;
  }
```

GameController - Game Loop

- Skips game physics and drawing when paused.
- Keeps requesting frames in the background.

```
// ...
  if (this.keys['esc']) {
    this.lastTime = time;
    this.animationFrameId =
requestAnimationFrame(frame);
    return;
  }
  // ...
```

GameController - Game Loop

- Calculates the time (step) to move characters fairly.
- Updates the Model and draws the new frame.

```
// ...
  const step = Math.min(time - this.lastTime, 100)
/ 1000;
  this.model.animate(step, this.keys);

  this.view.drawFrame({ step, /* ...DTOs... */ });
// ...
```

GameController - Game Loop

- Checks if the player has won or lost the level.
- Stops the current loop if the level is finished.

```
// ...
  const status =
this.model.getCurrentLevel().getStatus();
  if (status) {
    this.onLevelFinished(status);
    return;
  }
// ...
```

GameController - Game Loop

- Checks if the player has won or lost the level.
- Stops the current loop if the level is finished.

```
// ...
  const status =
this.model.getCurrentLevel().getStatus();
  if (status) {
    this.onLevelFinished(status);
    return;
  }
// ...
```

GameController - Game Loop

SCENARIO: Player loses and has NO lives left.

```
private onLevelFinished(status: string): void {
  this.view.unmount();
  if (status === 'lost') {
    this.model.loseLife();
    if (!this.model.getHasLivesRemaining()) {
      this.audioManager.play('gameOver');
      setTimeout(() => this.startLevel(0, 3), 0);
      return;
    }
    // ...
```

GameController - Game Loop

SCENARIO: Player loses but STILL HAS lives remaining.

```
// ... inside the 'lost' branch ...
    const level = this.model.reloadCurrentLevel();
    level.setLives(this.model.getLives());
    this.view.mount(level.getWidth(),
level.getHeight());
    this.runAnimation();
    return;
  }
  // ...
```

GameController - Game Loop

SCENARIO: Player wins the FINAL level of the game.

```
// ... after the 'lost' branch ...

  if (this.model.getIsLastLevel()) {
    this.audioManager.play('levelClear');
    setTimeout(() => this.startLevel(0, 3), 0);
    return;
  }
  // ...
```

GameController - Game Loop

SCENARIO: Player wins and ADVANCES to the next level.

```
// ... inside the 'won' branch ...

  const level = this.model.loadNextLevel();
  this.view.mount(level.getWidth(),
level.getHeight());
  this.runAnimation();
}
```

GameController - Game Loop

- Saves the current time for the next cycle.
- Requests the browser to run the next frame

```
// ...
  this.lastTime = time;
  this.animationFrameId =
requestAnimationFrame(frame);
};
```

Audio

Audio Manager

```
export type SoundType =
  | 'coin'
  | 'jump'
  | 'die'
  | 'levelClear'
  | 'gameOver';
```

Audio Manager

```
 constructor() {
  this.music = document.getElementById('music') as HTMLAudioElement;
  this.sound = document.getElementById('sound') as HTMLAudioElement;
  ...
}
```

Audio Manager

```
 constructor() {
  this.music = document.getElementById('music') as HTMLAudioElement;
  this.sound = document.getElementById('sound') as HTMLAudioElement;
  ...
}
```

```
<audio id="music"></audio>
<audio id="sound"></audio>
```

Audio Manager - SRP

```
constructor() {
  this.music = document.getElementById('music') as HTMLAudioElement;
  this.sound = document.getElementById('sound') as HTMLAudioElement;
  this.map.set('coin', 'assets/sound/coin.wav');
  this.map.set('jump', 'assets/sound/jump-small.wav');
  this.map.set('die', 'assets/sound/mario-die.wav');
  this.map.set('levelClear', 'assets/sound/stage_clear.wav');
  this.map.set('gameOver', 'assets/sound/gameover.wav');
}
```

Audio Manager - SRP

```
public play(soundType: SoundType) {
  this.sound.src = this.map.get(soundType)!;

  this.sound.play().catch((err: any) => {
    if (err?.name === ‘NotAllowedError’) {
      return;
    }
  });
}
```

Audio Manager - Event

```
playerTouched(type: string, actor?: Actor) {
    ...
  } else if (type === 'coin') {
    this.onSoundCallback?.('coin');
    ...
  }
}
```

Audio Manager - Event

Level.ts

```
onSoundCallback?: (soundType: SoundType) => void;
```

Audio Manager - Event

Level.ts

```
onSoundCallback?: (soundType: SoundType) => void;
```

GameModel.ts

```
public bindSoundEvent(callback: (sound: SoundType) => void)
{
  this.currentLevel!.onSoundCallback = callback;
}
```

Audio Manager - Event

Level.ts

```
onSoundCallback?: (soundType: SoundType) => void;
```

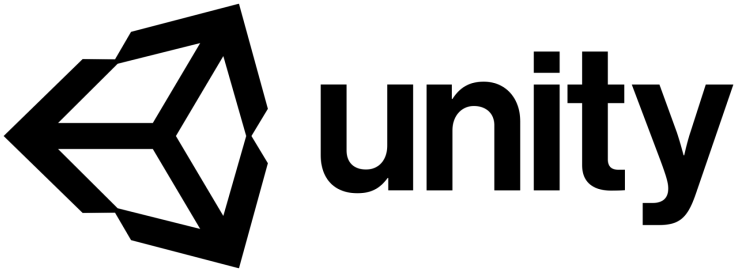
GameModel.ts

```
public bindSoundEvent(callback: (sound: SoundType) => void)
{
  this.currentLevel!.onSoundCallback = callback;
```

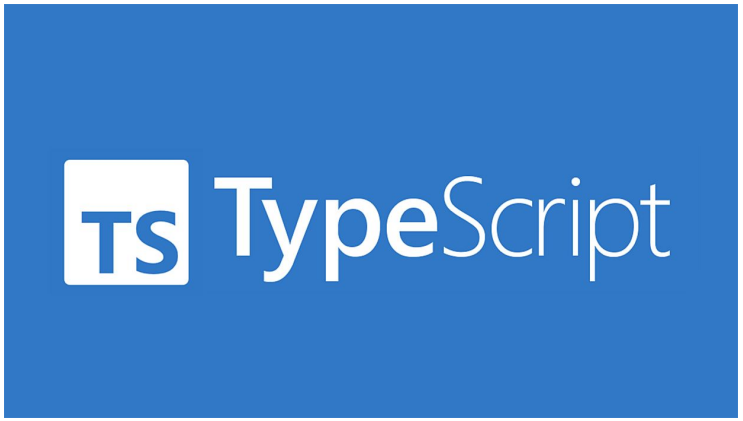
GameController.ts

```
  this.model.bindSoundEvent((soundType: SoundType) => {
    this.audioManager.play(soundType);
  });
```

Conclusions



VS



Any questions?

References

Core Project & Literature

- *Eloquent JavaScript (3rd Ed.)* – Chapter 16: Project: A Platform Game.
- *Platform Game Reference Implementation* – GitHub Repository (rubychi).

Tooling & Infrastructure

- *Gulp.js Official Documentation* – Build system and task automation.

Architecture & Quality Assurance (Peer Presentations)

- *MVC Design Pattern* – Class Presentation on Model-View-Controller architecture.
- *SonarQube Integration* – Class Presentation (ULL-ESIT-PAI 2025-2026).